Übungen Interaktives Spezifizieren und Verifizieren

Die mit ⋆ gekennzeichneten Übungen sind für „Experten“.

A Erste Begegnung mit Alloy

1. Java-Klassen

Gegeben sei folgende (überaus vereinfachte) Spezifikation des Konzepts der Klassen in Java.

```
sig Class {
    super : lone Class,
    mvars : set Var
}
one sig Object extends Class {}
sig Name {}
sig Var {
    name : one Name
}
```

Formulieren Sie folgende Eigenschaften als Fakten, die für Java-Klassen gelten müssen:

(a) Alle Klassen außer *Object* haben genau eine Basisklasse.

(b) Eine Klasse kann nicht von sich selbst erben.

(c) Die Namen der Member-Variablen einer Klasse müssen paarweise verschieden sein.

Erzeugen Sie Szenarien basierend auf dieser Spezifikation und finden Sie Konstellationen, die für Java-Klassen nicht gelten.

2. Zusammenhängende Graphen

Definieren Sie einen gerichteten Graphen in Alloy und erstellen Sie ein Prädikat, das besagt, dass der Graph zusammenhängend ist.

Lassen Sie den Alloy Analyzer Beispiele zusammenhängender Graphen erzeugen.

3. Vollständige Graphen

Definieren Sie einen Graphen in Alloy und erstellen Sie ein Prädikat, das besagt, dass der Graph vollständig ist, d.h. je zwei Ecken sind durch eine Kante verbunden.

Lassen Sie den Alloy Analyzer Beispiele von vollständigen Graphen erzeugen.

B Spezifikation struktureller Eigenschaften

4. Verwandtschaftsverhältnisse

Starten Sie mit folgender Spezifikation:

```
abstract sig Person {
  children: set Person,
  siblings: set Person
}

sig Man, Woman extends Person {}
```

```
sig Married in Person {
  spouse: one Married
}
```

(a) Erzeugen Sie Modelle zu dieser Spezifikation und überlegen Sie welche merkwürdigen Fälle auftreten können.

(b) Erweitern Sie die Spezifikation um solche Fälle auszuschließen.

(c) Formulieren Sie Annahmen und überprüfen Sie sie für folgende Punkte

- Es kann nicht sein, dass ein Elternteil einer Person gleichzeitig ein Geschwister ist.
- Wenn eine andere Person eine Geschwister ist, dann ist dies auch umgekehrt der Fall.
- Verheiratete Personen haben keinen gemeinsamen Vorfahr.

5. Algebraische Gesetze

Man kann in Alloy bestimmte Ausdrücke auf verschiedene Art ausdrücken. Wenn wir zum Beispiel in der Spezifikation einer Familie die binären Relationen `mother`, `father`, `sister` definiert haben, dann sind die Tanten einer Person `person`

```
person.mother.sister + person.father.sister
```

Dies kann man aber auch etwas kompakter ausdrücken, nämlich

```
person.(mother + father).sister
```

Überprüfen Sie durch mathematische Überlegung und mittels Alloy, ob für eine Menge `s` und binäre Relationen `p` und `q` die folgenden Gleichungen gelten:

(a) `s.(p + q) = s.p + s.q`

(b) `s.(p - q) = s.p - s.q`

(c) `s.(p & q) = s.p & s.q`

(d) `~(p + q) = ~p + ~q`

(e) `p.~q = q.p`

(f) `s.~p = p.s`

(g) `~p.~q = ~(q.p)`

6. Generalisierung/Spezialisierung in der Unified Modeling Language UML

In der Spezifikation der Unified Modeling Language werden für Generalisierungen bzw. Spezialisierungen in Table 9.1 auf Seite 121 Integritätsbedingungen definiert:

`{complete, disjoint}`: Indicates the generalization set is covering and its specific Classifiers have no common instances.

`{incomplete, disjoint}`: Indicates the generalization set is not covering and its specific Classifiers have no common instances.

`{complete, overlapping}`: Indicates the generalization set is covering and its specific Classifiers do share common instances.

`{incomplete, overlapping}`: Indicates the generalization set is not covering and its specific Classifiers do share common instances.

Spezifizieren Sie in Alloy Beispiele für die vier Möglichkeiten.

Zusatzfrage: Wie kann man diese vier Möglichkeiten in einer objekt-orientierten Sprache wie Java implementieren? Was ist ein wesentlicher Unterschied zwischen den relationalen Strukturen in Alloy und Klassen/Objekten in der Objekt-Orientierung?

7. Eigenschaften binärer Relationen

Gegeben sei eine Relation $r \subseteq A \times B$ (in Alloy: `r in A -> B`).

Man sagt

(a) die Relation ist *injektiv*, wenn zwei Elemente von A niemals dasselbe Element von B zugeordnet haben.

(b) die Relation ist *surjektiv*, wenn jedes Element von B mindestens einem Element von A zugeordnet ist.

(c) die Relation ist eine *partielle Funktion*, wenn jedes Element von A höchstens einem Element von B zugeordnet ist.

(d) die Relation ist *total*, wenn jedes Element von A mindestens einem Element von B zugeordnet ist.

(e) die Relation ist einen (totale) *Funktion*, wenn jedes Element von A genau einem Element von B zugeordnet ist.

(f) die Relation ist eine *Darstellung*, wenn sie partiell und injektiv ist.

(g) die Relation ist eine *Abstraktion*, wenn sie partiell und surjektiv ist.

(h) die Relation ist eine *Injektion*, wenn sie eine Funktion und injektiv ist.

(i) die Relation ist eine *Surjektion*, wenn sie eine Funktion und surjektiv ist.

(j) die Relation ist eine *Bijektion*, wenn sie eine Injektion und eine Surjektion ist.

Drücken Sie diese Eigenschaften durch *Multiplizitäten* in Alloy aus.

8. Instagram

Es gibt eine Webseite Alloy4Fun, auf der man den Alloy Analyzer auch online verwenden kann. Es gibt dort auch vorbereitete Aufgaben, von denen ich einige in diese Übungen übernommen habe.

Eine Spezifikation der Struktur einer Social Media Platform wie Instagram kann so aussehen (graphisch in Abb. 1):

```
sig User {
  follows : set User,
  sees : set Photo,
  posts : set Photo,
  suggested : set User
}

sig Influencer extends User {}

sig Photo {
  date : one Day
}
sig Ad extends Photo {}

sig Day {}
```

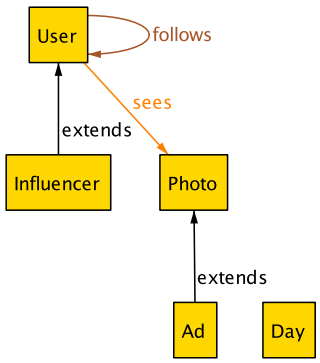


Abbildung 1: Struktur von Instagram

Die Aufgabe besteht darin, auf Basis dieser Sepzifikation Prädikate für folgende Aussagen in Alloy zu formulieren:

1. Every image is posted be one user.
2. An user cannot follow itself.
3. An user only sees (non ad) photos posted by followed users. Ads can be seen by everyone.
4. If an user posts an ad then all its posts should be labelled as ads.
5. Influencers are followed by everyone else.
6. Influencers post every day.
7. Suggested are other users followed by followed users, but not yet followed.
8. An user only sees ads from followed or suggested users.

Die Aufgabenstellung finden Sie auch auf Alloy4Fun Exercise Instagram. Sie können die Aufgaben in einer lokalen Installation des Alloy Analyzers lösen oder auch auf Alloy4Fun. Letzeres hat den Vorteil, dass Sie unmittelbar ein Feedback bekommen, ob die Lösung richtig ist.

Es gibt verschiedene Stile, in denen man Prädikate in Alloy formulieren kann. Sie können diese Aufgabe (und Alloy4Fun) auch dazu verwenden, mit diesen Stilen etwas zu experimentieren. Versuchen Sie also mehrere richtige Lösungen zu finden.

9. Hochschule

Entwerfen Sie eine Spezifikation der Situation an einer Hochschule, die folgende Gegebenheiten berücksichtigt:

- Es gibt Professoren, Dozenten und Studenten. Studenten sind entweder Bachelorstudenten oder Masterstudenten. Dozenten sind Professoren oder Masterstudenten. Studenten haben eine eindeutige Matrikelnummer und sind in einem Fachbereich eingeschrieben.
- Professoren betreuen Studenten.
- Studenten haben Kurse absolviert.

- Kurse werden von jeweils einem Dozenten angeboten. Kurse haben andere Kurse als Voraussetzung. In einen Kurs sind Studenten eingeschrieben, andere Studenten sind auf der Warteliste für den Kurs.
- Fachbereiche bieten Kurse an, von denen einige Pflichtkurse sind.
- Alle Dozenten sind Profs oder Masterstudenten.
- Jeder Fachbereich hat mindestens einen Dozenten.
- Pflichtkurse werden nur von Profs angeboten.
- Wenn kein Student in einen Kurs eingeschrieben ist, ist die Warteliste leer.
- Ein Student ist nicht zugleich eingeschrieben und auf der Warteliste sein.
- Ein Master kann nur Dozent eines Kurses sein, zu dem er sich nicht angemeldet hat.
- Es gibt keinen Zyklus bei den Voraussetzungen eines Kurses.
- Ein Student kann sich nur zu einem Kurs anmelden, dessen Voraussetzungen er hat.
- Der Mentor eines Masterstudenten betreut ihn auch.

Überprüfen Sie folgende Annahmen und korrigieren Sie ggfs. die Spezifikation:

- Ein Stuudent muss in mindestens einem Pflichtkurs eingeschrieben sein.
- Ein Student hat alle vorausgesetzten Kurse seiner absolvierten Kurse absolviert.
- Kein Dozent ist auf der Warteliste eines der Kurse, die er anbietet

Welche weiteren Integritätsbedingungen könnte man formulieren?

10. Beispiel Hochschule auf Alloy4Fun

Es gibt auf Alloy4Fun auch eine Aufgabe Hochschule. Vorgegeben ist folgende Struktur (graphisch in Abb. 2):

```
open util/ordering[Grade]

sig Person {
  teaches : set Course,
  enrolled : set Course,
  projects : set Project
}

sig Professor, Student in Person {}

sig Course {
  projects : set Project,
  grades : Person -> Grade
}

sig Project {}

sig Grade {}
```

Formulieren Sie Prädikate für folgende Aussagen. Sie können gerne wieder Alloy4Fun einsetzen – mit unmittelbarem Feedback.

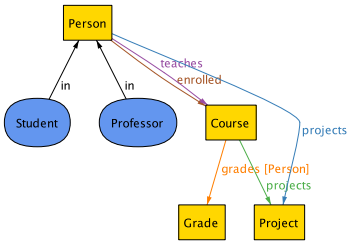


Abbildung 2: Struktur Hochschule auf Alloy4Fun

1. Only students can be enrolled in courses.
2. Only professors can teach courses.
3. Courses must have teachers.
4. Projects are proposed by one course.
5. Only students work on projects and projects must have someone working on them.
6. Students only work on projects of courses they are enrolled in.
7. Students work on at most one project per course.
8. A professor cannot teach herself.
9. A professor cannot teach colleagues.
10. Only students have grades.
11. Students only have grades in courses they are enrolled.
12. Students have at most one grade per course.
13. A student with the highest mark in a course must have worked on a project on that course.
14. A student cannot work with the same student in different projects.
15. Students working on the same project in a course cannot have marks differing by more than one unit.

11. Produktionsanlage

Noch eine Aufgabe aus Alloy4Fun. Es geht um die Struktur einer Produktionsanlage mit Arbeiten an Arbeitsplätzen, die aufeinander folgen und die von Menschen oder Robotern ausgeführt werden können.

Hier die Struktur (graphisch in Abb. 3):

```
sig Workstation {
  workers : set Worker,
  succ : set Workstation
}

one sig begin, end in Workstation {}

sig Worker {}

sig Human, Robot extends Worker {}
```

```
abstract sig Product {
  parts : set Product
}

sig Material extends Product {}

sig Component extends Product {
  workstation : set Workstation
}

sig Dangerous in Product {}
```

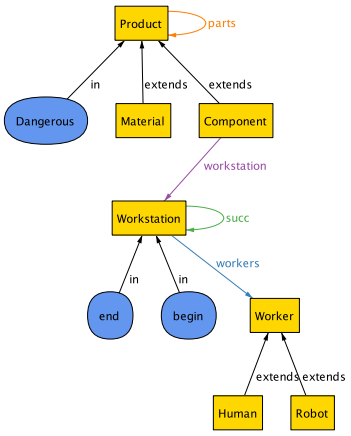


Abbildung 3: Struktur Produktionsanlage

Formulieren Sie Prädikate für folgende Aussagen:

1. Workers are either human or robots.
2. Every workstation has workers and every worker works in one workstation.
3. Every component is assembled in one workstation.
4. Components must have parts and materials have no parts.
5. Humans and robots cannot work together.
6. Components cannot be their own parts.
7. Components built of dangerous parts are also dangerous.
8. Dangerous components cannot be assembled by humans.
9. The workstations form a single line between begin and end.
10. The parts of a component must be assembled before it in the production line.

12. Erdkunde-Klassenarbeit

Logelei aus dem ZEIT-Magazin vom 24.05.2007

> Claudia musste für den Erdkundeunterricht vier Berge auswendig lernen. Am nächsten Tag in der Klassenarbeit kann sie sich noch an Folgendes erinnern:

Auf dem Felderer findet sich kein Steinhaufen.
Der Berg in Barunien heißt weder Schneehorn, noch ist er 2317 Meter hoch.
Auf einem der Berge war ein gigantischer Felsen.
Weder der Berg in Gorabien noch der Felderer ist 2581 Meter hoch.
Der Berg in Seborien ist 2128 Meter hoch.
Das Schneehorn ist nicht 2222 Meter hoch.
Der Berg mit der Hütte darauf ist weder 2222 Meter hoch noch der Borken, noch in Seborien.
Der Berg in Gorabien ist nicht 2317 Meter hoch.
Weder auf dem 2222 Meter hohen Berg noch auf dem Borken gibt es einen See. Es gab einen Weldberg.

Claudia grübelt und grübelt, aber mehr will ihr partout nicht einfallen. Da fällt ihr auf, dass die Tafel nicht geputzt ist. Dort steht noch, wohl von einem Erdkundeunterricht der Parallelklasse: „In Lusanien steht der Borken“.
Claudia ist erleichtert.

Wie heißen die vier Berge, wie hoch sind sie, in welchem Land sind sie, und welche Besonderheit findet man auf den vier Gipfeln?

Man kann die Frage noch erweitern: Gibt es nur eine Lösung? Gibt es mehrere Lösungen, wenn die Tafel geputzt gewesen wäre?

13. Das Zebra-Rätsel

Folgendes Rätsel (siehe Wikipedia) wird Einstein zugeschrieben, obwohl es keinerlei Hinweis darauf gibt, dass Einstein etwas damit zu tun hat.

Es geht so:

1 Es gibt fünf Häuser.
2 Der Engländer wohnt im roten Haus.
3 Der Spanier hat einen Hund.
4 Kaffee wird im grünen Haus getrunken.
5 Der Ukrainer trinkt Tee.
6 Das grüne Haus ist direkt links vom weißen Haus.
7 Der Raucher von Old-Gold-Zigaretten hält Schnecken als Haustiere.
8 Die Zigaretten der Marke Kools werden im gelben Haus geraucht.
9 Milch wird im mittleren Haus getrunken.
10 Der Norweger wohnt im ersten Haus.
11 Der Mann, der Chesterfields raucht, wohnt neben dem Mann mit dem Fuchs.
12 Die Marke Kools wird geraucht im Haus neben dem Haus mit dem Pferd.
13 Der Lucky-Strike-Raucher trinkt am liebsten Orangensaft.
14 Der Japaner raucht Zigaretten der Marke Parliaments.
15 Der Norweger wohnt neben dem blauen Haus.
16 Der Chesterfields-Raucher hat einen Nachbarn, der Wasser trinkt.

Dabei setzen wir voraus, dass jedes Haus genau eine der möglichen Eigenschaften hat und dass das verbleibendw Haustier das Zebra ist.

Die Frage ist dann:

„Wer trinkt Wasser und wem gehört das Zebra?“

Lösen Sie das Rätsel mit Alloy.

14. Das Damenproblem

Das Damenproblem besteht darin, n (üblicherweise 8) Damen so auf einem Schachbrett aufzustellen, dass sie sich nicht gegenseitig schlagen können.

Formulieren Sie das Damenproblem in Alloy und generieren Sie Lösungen dafür.

Hinweis: Wenn man in Alloy `Int` verwendet, dann muss man beachten, dass ganze Zahlen in Alloy explizit als Atome im untersuchten Universum erzeugt werden und zwar soviele wie die Anzahl der verwendeten Bits (`Bitwidth`) angibt. Bei arithmetischen Operationen mit `Int`s muss man außerdem wissen, dass `Int` *einstellige Relationen* sind und nicht *Zahlen*, siehe https://alloy.readthedocs.io/en/latest/modules/integer.html.

15. Kartenfärbungen

Planare Karten sollen so gefärbt werden, dass zwei benachbarte Länder nicht diesselbe Farbe haben.

(a) Erstellen Sie eine Spezifikation für Kartenfärbungen und lassen Sie den Alloy Analyzer beispielhafte Konfigurationen konstruieren.

(b) Zeigen Sie mittels dieser Spezifikation, dass die Karte der Staaten und Territorien Australiens mit drei Farben gefärbt werden kann.

(c) Geben Sie eine Konfiguration vor, bei der vier Farben erforderlich sind.

(d) Könnte es eine Karte geben, bei der man mehr als vier Farben benötigt?

16. Bipartiter Graph

Ein bipartiter Graph ist ein ungerichteter Graph, dessen Knoten in zwei disjunkte Teilmengen zerfallen, so dass zwischen den Knoten dieser Teilmengen keine Kanten verlaufen. Färben wir die Knoten der einen Teilmenge schwarz ein und die der adneren Teilmenge weiß, dann bedeutet das, dass jede Kante am einen Ende einen schwarzen und am anderen Ende einen weißen Knoten hat.

Spezifizieren Sie einen bipartiten Graphen, aber so, dass wir in der Visualisierung im Alloy Analyzer die Knoten auch als schwarz bzw. weiß darstellen können.

Sehen Sie auch ein Prädikat vor, das beschreibt, wann ein bipartiter Graph *vollständig* ist, d.h. dass jeder der schwarzen Knoten mit jedem der weißen Knoten verbunden ist.

17. Formeln der Aussagenlogik

Gesucht ist eine Alloy-Spezifikation, die Formeln der Aussagenlogik spezifiziert. Wir wollen voraussetzen, dass wir unsere Sprache der Aussagenlogik die Operatoren `Not`, `And`, `Or` sowie `Impl` hat.

18. Erfüllbare Formeln der Aussagenlogik

Erweitern Sie die Spezifikation der vorherigen Aufgabe so, dass nur erfüllbare Formeln erzeugt werden.

Hinweis: Das Modul `util/boolean` könnte hilfreich sein.

19. ⋆ Natürliches Schließen in der Aussagenlogik

Ausgehend von der Spezifikation von Formeln der Aussagenlogik ist es möglich, das natürliche Schließen in der Aussagenlogik in Alloy zu spezifizieren. Was ist zu tun?

(a) Man muss die Regeln des natürlichen Schließens als Prädikate formulieren.

(b) Für die Herleitung muss man eine rekursive Struktur vorsehen, deren Knoten jeweils einen Schritt der Herleitung beschreiben mit

- Verwendeter Regel,
- Voraussetzungen,
- Folgerung, und
- den verwendeten Teil-Herleitungen.

20. Gruppen

Eine Gruppe G ist ein Tupel $(G, *, 1)$ bestehend aus einer Menge von Elementen G, einer Funktion $* : G \times G \to G$ und einem ausgezeichneten Element $1 \in G$, so dass gilt:

(1) Inverse Elemente: zu jedem $x \in G$ gibt es ein $y \in G$ so dass gilt: $x * y = y * x = 1$

(2) Assoziativgesetz: für alle $x, y, z \in G$ gilt: $x * (y * z) = (x * y) * z$

(3) Neutrales Element: für alle $x \in G$ gilt: $1 * x = x * 1 = x$.

(a) Spezifizieren Sie die Struktur einer Gruppe in Alloy.

(b) Lassen Sie den Alloy Analyzer eine Gruppe mit drei Elementen generieren.

(c) Überprüfen Sie die Aussage, dass jedes Element einer Gruppe ein *eindeutiges* inverses Element hat mittels des Alloy Analyzers.

(d) Schreiben Sie eine `assert`-Anweisung zur Überprüfung, ob eine Gruppe *kommutativ* ist, d.h. ob für alle $x, y \in G$ gilt: $x * y = y * x$. Überprüfen Sie die Anweisung mit steigendem *Scope* beginnend bei 1.

21. ⋆ Gruppenaxiome

In einem einführenden Aufsatz über das Suchen endlicher Modelle erwähnt Jian Zhang folgendes Beispiel:

> It had been thought that the follwing equation alone can axiomatize the group theory
> $i(f(f(x, f((f(y, z), i(f(y, f(i(u), i(z))))))), x)) = u$
> Here i means inverse, f means multiplication, and all the variables are universally quantified.
> [Jian Zhang: Finite Model Searching: An Introduction and Some Personal Notes, Beijing 2002, Aug.]

Der Autor nennt ein Beispiel einer Struktur mit 2 Elementen, die die Gleichung erfüllt, aber keine Gruppe ist. McCune hat diese mit einem „Modelfinder“ gefunden.

Wie geht das mit Alloy? Bedenken Sie aber auch, dass es unterschiedliche Möglichkeiten gibt in Alloy die Gruppenaxiome zu spezifizieren.

22. Projektive Ebenen

Eine projektive Ebene ist eine Menge von Punkten und Geraden mit folgenden Eigenschaften:

(1) Zu je zwei verschiedenen Punkten gibt es genau eine Gerade, auf der die Punkte liegen.

(2) Zu je zwei verschiedenen Geraden gibt es genau einen Punkt, in dem sich die Geraden schneiden.

(3) Auf jeder Geraden liegen wenigstens 3 Punkte.

(4) Durch jeden Punkt gehen mindestens 3 Geraden.

- Spezifizieren Sie die Struktur einer (endlichen) projektiven Ebene in Alloy.
- Zeigen Sie mit Alloy, dass die kleinste endliche projektive Ebene 7 Punkte und 7 Geraden hat.
- Können Sie mit Alloy auch eine weitere endliche projektive Ebene mit mehr als 7 Punkten finden? (Vorsicht: Die Suche kann etwas dauern :-))

23. Königsberger Brücken

Die Stadt Königsberg wird durch den Fluss Pregel mit seinen beiden Armen geteilt, dadurch entstehen zwei „Inseln“. Die beiden Stadthälften sind durch je drei Brücken mit den Inseln verbunden. Die Inseln sind untereinander durch eine weitere Brücke verbunden, wie in Abb. 4 dargestellt.

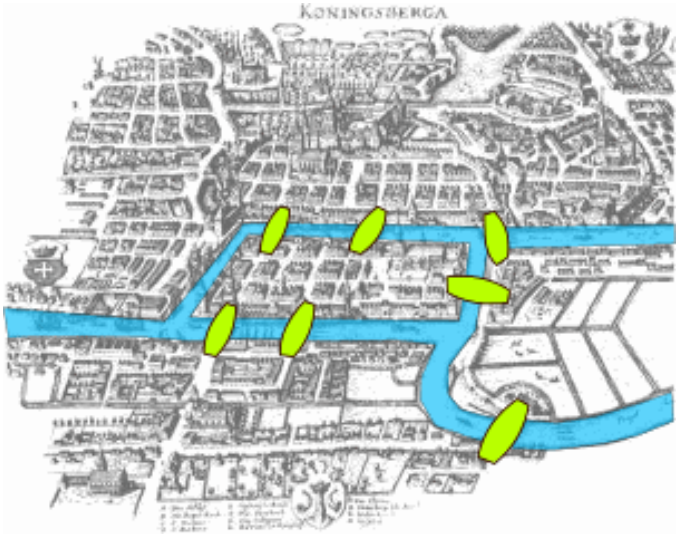


Abbildung 4: Königsberger Brücken

Leonhard Euler bewies, dass es keinen Weg gibt, auf dem man alle sieben Brücken genau einmal überquert.

Sein Argument geht so: Zu jedem Gebiet gibt es eine ungerade Zahl von Brücken. Damit es einen Weg gibt, der alle Brücken genau einmal überquert, dürfte es maximal 2 Gebiete mit einer ungeraden Anzahl von Brücken geben, nämlich das, wo der Weg beginnt und das, wo er endet. Die anderen Gebiete müssten eine gerade Anzahl von Brücken haben, weil man ja auf zwei verschiedenen Brücken das Gebiet betreten und wieder verlassen muss.

Wie kann man das Problem in Alloy formulieren und dann zeigen, dass ein „Eulerscher“ Weg nicht existiert?

24. Gefängnis

Folgende Aufgabe ist aus dem Buch von Daniel Jackson *Software Abstractions*:

A program is needed to assign inmates to cells in a prison. The assignment must avoid placing two inmates in the same cell if they are members of different gangs.

Here is a suitable template:

```
module prison

sig Gang {members: set Inmate}
sig Inmate {room: Cell}
sig Cell {}

pred safe () {
... your constraints here
}

pred show () {
... your constraints here
}
run show
```

(a) Complete the predicate *safe* characterizing a safe assignment, and generate examples of both safe and unsafe assignments by running the simulation predicate show, with appropriate invocations of safe as its constraint.

(b) Write a new predicate called *happy*, saying that gang members only share cells with members of the same gang. Does this condition always follow from a safe assignment? By writing an assertion and a command to check it, find a counterexample, and explain why not.

(c) Add a constraint as a fact that ensures that safety will indeed imply happiness. Run the simulation predicate to make sure that you haven't introduced an inconsistency, and check the assertion again to make sure it now has no counterexample.

C Spezifikation von Dynamik

25. Uhr in Alloy

Spezifizieren Sie eine Uhr, die die Stunden 1 bis 12 anzeigen kann und lassen Sie sie laufen.

26. Bankkarten

Gegeben sei folgende Spezifikation eines Systems einer Bank zur Ausgabe von Kreditkarten (das Beispiel stammt von Alexandre Abreu):

```
abstract sig Status {}

one sig Unissued, Issued, Cancelled extends Status {}

sig Card {
  var status : one Status
}

sig Client {
  var cards : set Card
}
```

Erweitern Sie die Spezifikation folgendermaßen:

(a) Zu Beginn soll kein Kunde eine Karte besitzen.

(b) Karten können einem Kunden ausgegeben werden.

(c) Eine Karte kann gesperrt werden (Cancelled).

Überprüfen Sie die beiden folgenden Eigenschaften des spezifizierten Systems:

(a) Ein Kunde kann niemals eine nicht ausgegebene (Unissued) Karte besitzen.

(b) Eine Karte kann niemals gleichzeitig mehr als einem Kunden gehören.

Spielen Sie mit der Spezifikation, indem Sie Szenarien generieren lassen.

27. Flussüberquerungsrätsel

Ein Bauer steht mit einem Wolf, einer Ziege und einem Kohlkopf auf einer Seite eines Flusses und möchte auf die andere Seite mit einem Boot übersetzen. Das Boot kann aber außer ihm selbst nur noch einen weiteren Passagier aufnehmen. Das Problem: Wolf und Ziege können nicht unbeaufsichtigt bleiben, der Wolf würde die Ziege fressen. Genausowenig können Ziege und Kohlkopf unbeaufsichtigt bleiben, weil die Ziege dann den Kohl verschlingen würde.

Wie gelingt es dem Bauern, den Fluss zu überqueren, ohne dass ein Unglück passiert? Lösen Sie das Rätsel mit dem Alloy Analyzer.

28. Chamäleons

Spezifizieren Sie eine Welt mit einer fixen Anzahl von Chamäleons, die jeweils eine von den Farben grün, blau und gelb haben können. Wie Chamäleons so sind, wechseln sie die Farbe. In unserer (fiktiven) Welt verhält es sich mit dem Farbwechsel so: Immer wenn sich zwei Chamäleons mit verschiedenen Farben treffen, wechseln sie ihre Farben: eines wechselt in die dritte Farbe, das andere übernimmt die Farbe des Gegenübers.. Dies ist auch die einzige Siutation, in der sie ihre Farbe wechseln. (Auch diese Aufgabe stammt von Alexandre Abreu)

Überprüfen Sie folgende Eigenschaften der Welt der Chamäleons:

(a) Wenn die Chamäleons irgendwann alle die gleiche Frabe haben, dann gibt es nie wieder einen Farbwechsel.

(b) Wenn es am Anfang ein grünes Chamäleon gibt, aber kein blaues, dann ist es unmöglich, dass alle irgendwann gelb sind.

(c) Gibt es ein Szenario, in dem ein Chamäleon alle möglichen Farben sukzessive annimmt und das immerzu?

29. Tic-Tac-Toe — Drei Gewinnt

Auf einem quadratischen 3×3 Felder großen Spielfeld setzen zwei Spieler abwechselnd ihr Zeichen (ein Spieler Kreuze, der andere Kreise) in ein freies Feld. Der Spieler, der als Erster drei Zeichen in eine Zeile, Spalte oder Diagonale setzen kann, gewinnt.

Spezifizieren Sie das Spiel in Alloy.

Zusatzaufgabe: Zeigen Sie, dass es möglich ist, dass das Spiel unentschieden endet. Das heißt, alle neun Felder sind gefüllt, ohne dass ein Spieler die erforderlichen Zeichen in einer Reihe, Spalte oder Diagonalen setzen konnte.

30. Variante des Echo-Algorithmus

In Erste Begegnung mit Alloy 6 haben wir gesehen, wie man mit dem Echo-Algorithmus den aufspannenden Baum eines Graphen konstruieren kann.

In dieser Spezifikation haben wir ein Feld `color` verwendet, das `Green` wird, wenn ein Knoten sein Echo zurückgesandt hat. In der Visualisierung kann man eine Funktion spezifizieren, mit der man die Knoten auch entsprechend im Visualisierungsfenster grün anzeigen kann.

Das geht aber auch anders: Verändern Sie den Algorithmus aus Abschnitt 6 von Erste Begegnung mit Alloy 6 so, dass man die Knoten tatsächlich in der Visualisierung grün anzeigen kann, jedoch nicht durch eine Funktion. Tipp: Die Spezifikation einer Teilmenge der Knoten, die ihr Echo geschickt haben, könnte dabei helfen.

31. Aufspannender Baum eines Graphen

In Erste Begegnung mit Alloy 6 haben wir gesehen, wie man mit dem Echo-Algorithmus den aufspannenden Baum eines Graphen konstruieren kann. Dieser Algorithmus endet damit, dass das Echo der Nachrichten zum Initiator zurückgekommen ist. Der aufspannende Baum ist doch dann längst konstruiert.

Passen Sie die Variante der Spezifikation aus Abschnitt 6 von Erste Begegnung mit Alloy 6 so an, dass kein Echo zurückgeschickt wird. Es sollte dann möglich sein, den Spannbaum in $n+1$ Schritten für Graphen mit n Knoten zu konstruieren.

32. Verschönerung von diehard

Verwenden Sie die am Beispiel des Echo-Algorithmus vorgestellte Mimik, um das Beispiel von „diehard“ so zu erweitern, dass nicht nur der Ablauf angezeigt wird, sondern jeweils auch welche Transition ausgeführt wird.

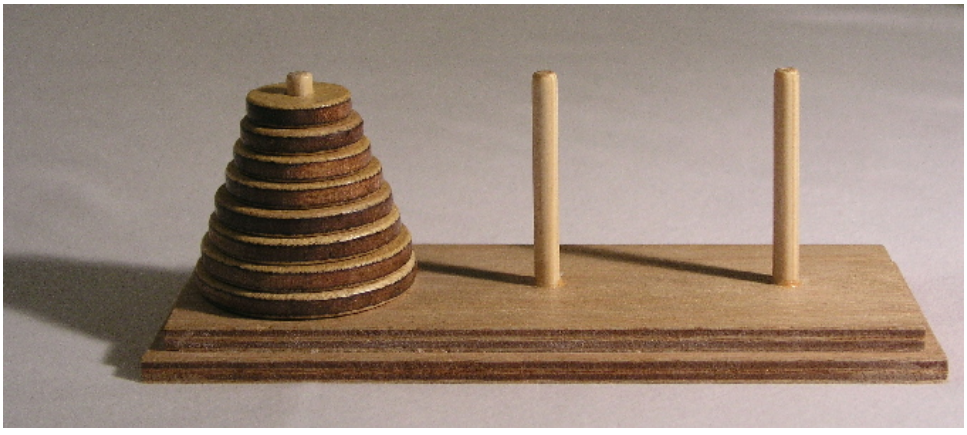


Abbildung 5: Türme von Hanoi, von https://commons.wikimedia.org/wiki/File:Tower_of_Hanoi.jpeg

33. Türme von Hanoi

Das Spiel „Türme von Hanoi“ besteht aus drei gleich großen Stäben, den Türmen. Auf die Türmen können mehrere gelochte Scheiben gelegt werden, die alle verschieden groß

sind. Zu Beginn liegen alle Scheiben auf dem linken Stab, und zwar der Größe nach geordnet, die kleinste Scheibe oben.

Ziel des Spiels ist es, den kompletten Scheiben-Stapel vom linken Turm auf den rechten Turm zu versetzen. Dabei darf bei jedem Zug die oberste Scheibe eines Turms auf einen anderen Turm befördert werden, allerdings so, dass jeder Turm immer der Größe nach geordnet ist.

Lösen Sie die Aufgabe mit Alloy.

34. Königsberger Brücken mit LTL

Die Aufgabe zu den Königsberger Brücken kann man auch mit LTL lösen. Zeigen Sie damit, (a) dass es keinen Weg gibt, auf dem alle sieben Brücken nur genau einmal überquert werden dürfen und (b) dass es natürlich Wege über alle Brücken gibt, wenn man diese Voraussetzung weglässt.

35. Speisende Philosophen

Fünf Philosophen sitzen an einem runden Tisch mit jeweils einer Gabel zwischen ihnen und einem Teller köstlicher Speise vor ihnen. Ein Philosoph darf essen, wenn er beide Gabeln, die linke und die rechte zwischen ihm und seinen beiden Nachbarn in Händen hält (siehe Wikipedia zum Philosophenproblem).

Spezifizieren Sie die Situation und zeigen Sie, dass es (1) eine Verklemmung geben kann, aber auch (2) einen Ablauf, an dem wenigstens ein Philosoph die köstliche Speise zu sich nehmen kann. Können Sie (3) auch einen Ablauf erzeugen, bei dem jeder Philosoph wenigstens einmal zugreifen konnte?